Models

MODALIDADE	DURAÇÃO ESTIMADA	FORMATO DE ENTREGA
Dupla	**2 aulas (90 min)**	**PDF + link do código**

Nas aulas anteriores vimos o padrão MVT (Model–View–Template) do Django e como servir Templates com arquivos estáticos. Falta a peça que conecta tudo: o Model, que descreve os dados da aplicação, e a View, que busca esses dados e decide o que mandar para a tela. Esta atividade é uma pesquisa guiada: em vez de uma aula expositiva, vocês vão investigar a documentação oficial, testar código e responder às perguntas abaixo antes de aplicar o que aprenderam num desafio prático.

> **Observação:** *Respondam com suas próprias palavras. O objetivo é registrar o que vocês entenderam da pesquisa, não copiar trechos da documentação.*

1. Investigando os Models

1.1. O que é um Model no Django e qual problema ele resolve dentro da arquitetura MVT?

1.2. O que é ORM (Object-Relational Mapping) e como o Django usa esse conceito para evitar escrever SQL manualmente?

1.3. Como se declara um Model? Liste pelo menos 5 tipos de campo e explique quando usar cada um.

1.4. Para que serve a classe **Meta** dentro de um Model? Cite pelo menos duas opções configuráveis nela.

1.5. O que são migrations? Explique a diferença entre `makemigrations` e `migrate`.

2. Ligando Models às Views

2.1. Como uma view importa e utiliza um Model para buscar dados do banco?

2.2. O que é um QuerySet? Explique a diferença entre `Model.objects.all()`, `.filter()` e `.get()`.

2.3. Como os dados buscados no Model chegam até o Template? Pesquise o dicionário de contexto passado para `render()`.

2.4. Pesquise o atalho `get_object_or_404`. Por que ele é preferível a um `Model.objects.get()` direto dentro de uma view?

3. Desafio prático · Mini biblioteca

Com o que vocês pesquisaram, construam uma aplicação mínima que lista livros cadastrados no banco.

Especificação do Model Livro

Campo	Tipo sugerido	Observação
titulo	CharField	max_length=200
autor	CharField	max_length=120
ano_publicacao	IntegerField	—
disponivel	BooleanField	default=True

Passo a passo

1. Crie um app biblioteca.

2. Defina o Model Livro.

3. Rode as migrações.

4. Cadastre pelo menos 3 livros pelo Django Admin ou pelo shell.

5. Crie a view listar_livros que busca todos os livros com o ORM e os envia para um template.

6. Criar o template lista_livros.html exibindo título, autor e ano de cada livro.

7. Registrar a URL da view e testar a página no navegador.

Entrega

- **Documento —** Respostas das Partes 1 e 2, em PDF ou documento compartilhado.
- **Código —** Link do repositório (GitHub) com models.py, views.py e o template.
- **Evidência —** Uma captura de tela da página com a lista de livros funcionando.
- **Canal —** Enviar pelo classroom.